

# Rapport : Méthode directe et stockage bande

SY Aboubacar

7 janvier 2026

## Exercice 5 : DGBTRF, DGBTRS, DGBSV

### DGBTRF - Factorisation LU

**Fonction** : Effectue la factorisation LU d'une matrice bande.

**Opération** :

$$A = L \cdot U$$

où  $L$  est une matrice triangulaire inférieure et  $U$  une matrice triangulaire supérieure.

**Format** : Stocke les matrices  $L$  et  $U$  dans le tableau d'entrée **AB**.

**Utilisation** :

```
1 dgbtrf_(&la, &la, &kl, &ku, AB, &lab, ipiv, &info);
```

**Complexité** :

- arithmétique :  $\mathcal{O}(n \cdot (kl * ku))$  pour une matrice bande de diagonale  $n$ . Car pour chaque colonne on va : calculer le/les pivot (kl division), le soustraire à ( $ku$ ) éléments à droite.
- mémoire : étant donné qu'on a réécrit dans la même matrice, la mémoire n'augmente pas donc :  $\mathcal{O}(n \cdot (kl + ku))$

### DGBTRS - Résolution du système

**Fonction** : Résout un système linéaire  $A\mathbf{x} = \mathbf{b}$  après factorisation.

**Nécessite** : Doit être appelé après une factorisation LU, soit DGBTRF ou notre implémentation de la factorisation LU tridiagonale.

**Opération** :

$$\begin{cases} L\mathbf{y} = P\mathbf{b} & (\text{descente}) \\ U\mathbf{x} = \mathbf{y} & (\text{remonter}) \end{cases}$$

**Utilisation** :

```
1 dgbtrs_("N", &la, &kl, &ku, &NRHS, AB, &lab, ipiv, RHS, &la, &info);
2 // AB -> matrice LU
3 // RHS -> b
4
```

**Complexité** :

- arithmétique :  $\mathcal{O}(\text{descente} + \text{remonte}) \rightarrow \mathcal{O}(n * kl + n * ku)$  car pour chaque ligne il peut y avoir au maximum kl/ku éléments. Pour une matrice bande de diagonale  $n$ ,  $kl$  nombre de sous-diagonale et  $ku$  nombre de sur-diagonale.
- mémoire : étant donné qu'on a écrit dans la même matrice, la mémoire n'augmente pas donc :  $\mathcal{O}(n * (kl + ku))$

## DGBSV - Résolution complète

**Fonction :** Résout un système linéaire en faisant une factorisation LU.

**Opération :**

$$\text{DGBSV} = \text{DGBTRF} + \text{DGBTRS}$$

$$AX = B$$

**Utilisation :**

```
1 dgbsv_(&n, &kl, &ku, &nrhs, AB, &ldab, ipiv, RHS, &ldb, &info);
2 // AB -> matrice bande d'ordre n, avec ku sur-diagonales et kl sous-diagonales
3 // RHS -> b
4
```

**Complexité :** Même que DGBTRF + DGBTRS.

- arithmétique :  $\mathcal{O}(\text{DGBTRF} + \text{DGBTRF}) = \mathcal{O}(n * (kl + ku) + n * kl + n * ku) = \mathcal{O}(n * (2kl + 2ku))$  pour une matrice bande de diagonale  $n$ .
- mémoire : étant donné qu'on réécrit dans la même matrice, la mémoire n'augmente pas donc :  $\mathcal{O}(n * (kl + ku))$

## 2. Évaluation des performances et complexité

| Taille $n$ | Temps DGBTRF (s) | Temps DGBTRS (s) | Temps DGBSV (s) | Erreur résiduelle         |
|------------|------------------|------------------|-----------------|---------------------------|
| 10         | 0.000079         | 0.000003         | 0.000098        | $5.249507 \times 10^{-8}$ |
| 100        | 0.000122         | 0.000010         | 0.000077        | $1.207707 \times 10^{-6}$ |
| 10000      | 0.000698         | 0.000569         | 0.002051        | $1.052995 \times 10^{-4}$ |
| 100000     | 0.004275         | 0.003620         | 0.007898        | $1.673769 \times 10^{-3}$ |

TABLE 1 – Performances des fonctions LAPACK

### Analyse de complexité :

On observe une évolution de temps linéaire pour DGBTRF et DGBTRS le temps d'exécution augmente proportionnellement à la taille  $n$  du problème, confirmant les complexité théorique. Pour DGBSV on remarque que pour les matrices de petite dimension elle est plus rapide de DGBTRF + DGBTRS, mais devient moins performante pour les matrices de grande dimension. L'erreur résiduelle croît nécessairement à cause de l'accumulation des erreurs d'arrondi lors des opérations en précision flottante

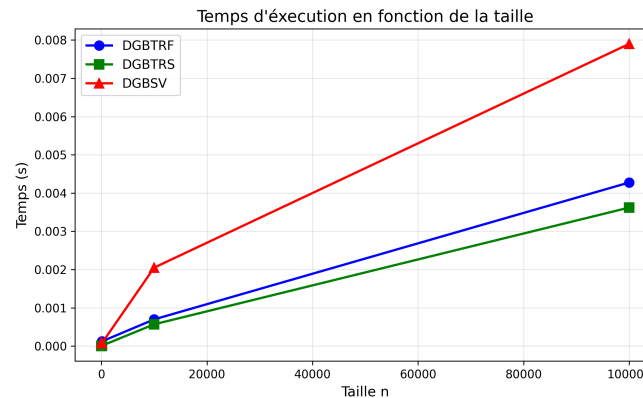


FIGURE 1 – Temps d'exécution en fonction de la taille  $n$

## Exercice 6 : LU pour matrices tridiagonales

### 1. Implémentation de la factorisation LU

Pour une matrice tridiagonale  $A$  (format GB avec  $kl = ku = 1$ ), l'algorithme de factorisation LU spécialisé est :

```
1 int indexABCol(int i, int j, int *lab){
2     // TODO: Return the correct index formula for column-major band storage
3     return (i-j)+(*lab)*j +2;
4 }
5
6 int dgbtrftridiag(int *la, int*n, int *kl, int *ku, double *AB, int *lab, int *
7     ipiv, int *info){
8     // TODO: Implement specialized LU factorization for tridiagonal matrices
9
10    for(int i=0;i<(*la);i++){
11        ipiv[i] = i+1;
12    }
13
14    double e = 1e-12;
15    int _n = *n;
16    for(size_t i =1;i<_n;i++){
17
18        double mi = AB[indexABCol(i,i-1,lab)] / AB[indexABCol(i-1,i-1,lab)];
19        AB[indexABCol(i,i-1,lab)] = mi; // mi -> multiplicateur
20
21        AB[indexABCol(i,i,lab)] = AB[indexABCol(i,i,lab)] - AB[indexABCol(i-1,i,lab)]
22        *mi;
23    }
24
25    *info = 0;
26    return *info;
27 }
```

L'algorithme implémente une factorisation  $LU$  optimisée pour les matrices tridiagonales au format bande LAPACK. Pour chaque élément  $a_{i,i-1}$  de la sous-diagonale L, il calcule le multiplicateur  $m_i = \frac{a_{i,i-1}}{a_{i-1,i-1}}$  qu'il stocke à sa place. L'élément diagonal est ensuite mis à jour selon  $u_{i,i} = a_{i,i} - a_{i-1,i} \times m_i$ .

La complexité de cette algorithme : puisqu'on parcourt uniquement la diagonale et que pour chaque valeur on fait 1 division, 1 soustraction, 1 multiplication, la complexité sera linéaire de l'ordre :  $\mathcal{O}(3(n-1))$

### 2. Méthode de validation

Lorsque la solution analytique exacte  $\mathbf{x}_{\text{exact}}$  est disponible, une validation peut être proposée en effectuant le calcul de l'erreur relative :

$$e_{\text{rel}} = \frac{\|\mathbf{x}_{\text{calc}} - \mathbf{x}_{\text{exact}}\|}{\|\mathbf{x}_{\text{exact}}\|}$$

Puisqu'on a nos disposition la fonction DGBSV, la solution obtenue avec DGBSV peut être utilisée comme  $\mathbf{x}_{\text{exact}}$ . L'algorithme est alors considéré comme validé lorsque l'erreur relative est inférieure à une tolérance fixée  $t$  :

$$e_{\text{rel}} < t \Rightarrow \text{Algorithme validé}$$

### 3. Évaluation des performances et comparaison

| Taille $n$ | LU triag (s) | DGBSV (s) | Erreur LU triag            | Erreur DGBSV               |
|------------|--------------|-----------|----------------------------|----------------------------|
| 1000       | 0.000057     | 0.000169  | $1.054400 \times 10^{-13}$ | $1.054400 \times 10^{-13}$ |
| 5000       | 0.000398     | 0.000714  | $1.759042 \times 10^{-12}$ | $1.759042 \times 10^{-12}$ |
| 10000      | 0.000403     | 0.000838  | $2.759671 \times 10^{-12}$ | $2.759671 \times 10^{-12}$ |
| 50000      | 0.001821     | 0.004162  | $6.380102 \times 10^{-11}$ | $6.380102 \times 10^{-11}$ |
| 100000     | 0.003691     | 0.008826  | $3.233329 \times 10^{-10}$ | $3.233329 \times 10^{-10}$ |

TABLE 2 – Comparaison des performances et précision entre notre implémentation maison et DGBSV

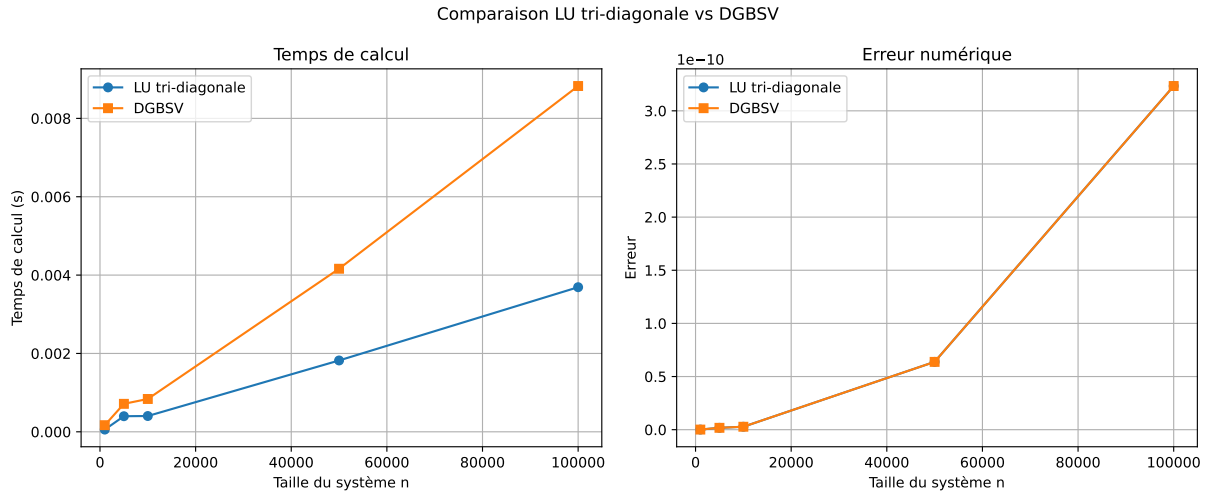


FIGURE 2 – Comparaison des performances et de la précision numérique entre l'implémentation LU tri-diagonale maison et DGBSV de LAPACK : évolution des temps de calcul (à gauche) et des erreurs numériques (à droite) en fonction de la taille du système  $n$ .

Sur le graphique de l'erreur numérique, étant donné que les erreurs sont identiques, les deux courbes se superposent.

### 4. Analyse des performances

On observe que, pour des tailles de problème croissante, l'implémentation LU tri-diagonale maison est plus rapide que la fonction DGBSV. L'écart de temps de calcul est d'environ un facteur 2 et reste le même lorsque la taille du système augmente. Cela peut s'expliquer par le fait que l'implémentation maison exploite directement la structure tri-diagonale de la matrice, tandis que DGBSV est une fonction plus générale pour matrices bandes, impliquant un surcoût algorithmique et mémoire.

Les temps de calcul augmentent de manière linéaire avec la taille  $n$  pour les deux méthodes, ce qui correspond à la complexité attendue pour la résolution de systèmes linéaires à bande. Toutefois, le facteur constant reste plus faible pour la méthode LU tri-diagonale, ce qui devient particulièrement visible pour les grandes tailles de problème.

Les erreurs numériques obtenues avec les deux méthodes sont identiques pour toutes les tailles testées. Elles augmentent progressivement avec  $n$ , passant de l'ordre de  $10^{-13}$  à  $10^{-10}$ , ce qui est expliqué avec les effets d'accumulation des erreurs d'arrondi en arithmétique double précision. L'implémentation LU maison est aussi précise et stable numériquement que la fonction `DGBSV` pour ce type de matrices.

## Conclusion

Ces résultats montrent qu'une implémentation spécialisée exploitant la structure tri-diagonale permet d'obtenir de meilleures performances avec la même précision par rapport à une fonction générique comme `DGBSV`.